Aula 4 — PWA Offline-First

"Instale uma vez, funcione em qualquer lugar — mesmo sem internet"

Prof. Jackson Smith Moisés Matias
PUC Minas IEC · 19/10/2026

Recap da Aula 3 — o que a gente fez

- **Flutter vs React Native:** UI desenhada pelo próprio Flutter (Impeller/GPU) × componente nativo; isolates × event loop
- **Dart + widgets:** tudo é widget · Stateless × Stateful · `setState` · estado com Riverpod (prop drilling → Provider → Riverpod)
- **Offline-first em camadas:** a UI só conhece o *estado*; só a camada de dados sabe se veio do disco ou da rede (o **cartão de embarque** funciona sem sinal) · demo em modo avião
- **Atividade 3** (Flutter + Firebase + offline-first): **fecha hoje 23:59** — o bot ainda aceita PR tardio, e quem precisar de mais prazo avisa no comentário do PR

> Hoje a mesma ideia de offline-first, **na web**: quem faz o papel de "disco" e de "camada de dados" no navegador?

Ponte: o offline-first que você fez no Flutter → PWA

Atividade 3 (Flutter)	Hoje (web / PWA)
T10 — persistência offline do Firestore	**IndexedDB** (`idb`) guarda os dados
T11 — `OfflineBanner`	hook `useOfflineStatus` (**TODO 5** da A4)
T13 — repositório cache-first / SWR	`moviesRepository` + **Workbox** (NetworkFirst, SWR)
T14 — TTL do cache	`expiration.maxAgeSeconds` no Workbox
T15 — `SyncQueue` (fila + flush)	**Background Sync** / outbox
App abre sem rede	**Service Worker** + Cache Storage servem o app shell

> Mesma arquitetura, outra plataforma. O que muda: no web *você* escreve (ou configura) o proxy de rede — o Service Worker.

Agenda da aula (3h30)

Bloco 1 (~100min) — PWA: conceito → demo → Service Worker → manifest/instalação → mão na massa
→ Anatomia · Demo Flutter Shell · lifecycle do SW · estratégias de cache · setup A4 + TODO 1

Intervalo (30min)

Bloco 2 (~80min) — Offline-first em profundidade + decisão de arquitetura
→ Workbox/ `vite-plugin-pwa` · IndexedDB · atualização do app · Background Sync · PWA × RN-Web × Capacitor · A4

> KMP fica para a **Aula 5 (26/10)**, junto com BFF/GraphQL.

Objetivos de aprendizagem

Desenvolvimento de PWA:

- Explicar o ciclo de vida do Service Worker (install → waiting → activate → controlling) e o que acontece em cada fase
- Escolher a estratégia de cache por tipo de recurso (Cache-first, Network-first, SWR) e configurá-la com Workbox / `vite-plugin-pwa`
- Decidir **como o usuário recebe uma nova versão** do app (autoUpdate × prompt)
- Tornar uma SPA instalável (manifest + `beforeinstallprompt`) e usável offline (Cache Storage + IndexedDB)
- Comparar PWA × RN-Web × Capacitor × nativo e justificar a escolha

Hybrid shell (bônus): PWA dentro de um shell Flutter/RN (WebView)

"PWA é só web?" — fato vs ficção

Twitter Lite *(Eng Blog, 2017)*: offline real, 75% mais tweets enviados.
Pinterest *(web.dev, 2017)*: 60% mais engajamento, instalável.
Spotify Web *(2023)*: PWA instalável no desktop — mesmo conteúdo do app.
Forbes *(2016)*: 100ms de melhoria no TTI → +1% de sessões.

> **PWA não é cosmético.** Mas não substitui nativo em todos os casos — limitações reais no iOS existem (sem push até iOS 16.4, sem Background Sync). Próximos slides detalham.

> ⚠️ Dados acima: blogs de engenharia das próprias empresas (não estudos controlados independentes).

🧭 Missão — site comum × PWA (10 min, no seu celular)

Abra os dois, um de cada vez, e compare:

	Site comum	PWA — `pwa-tmdb-aula4.vercel.app`
Exemplo	themoviedb.org	o app da aula

1. **Carregue** o PWA com internet, navegue um pouco (role a lista, abra a aba ★)
2. Ligue o **modo avião** e **recarregue** → o que acontece em cada um?
3. **Feche o navegador** (com o avião ligado) e abra de novo
4. **Instale** o PWA (Android/Chrome: menu → *Instalar app* · iPhone: Compartilhar → *Adicionar à Tela de Início*) e abra pelo ícone → tem barra de endereço?
5. Notebook: **DevTools → Application** → Service Workers · Cache Storage · IndexedDB · Manifest

> **Anote 3 diferenças** que você *viu* (não as que leu). Vamos comparar no chat — e elas voltam no `ENTREGA.md` da A4.

Missão — o que você deveria ter visto

Observação	Por quê (o que está por baixo)
PWA abre offline; o site comum mostra "sem internet"	**Service Worker** serve o *app shell* do Cache Storage
Lista de filmes aparece offline	**IndexedDB** guarda os dados · estratégia NetworkFirst
Ícone na tela inicial, sem barra de endereço	**Manifest** (`display: standalone`)
Application → "activated and is running"	ciclo de vida do SW (vimos acima)

> Um **PWA é um site** — com SW + manifest. O que muda é o que o *navegador* passa a fazer por ele. Quem decide isso é o **seu código**: é o que a A4 pede.

Missão — o que varia de aparelho pra aparelho (achados reais)

O que apareceu	Por quê
Botão compartilhar abre o menu do **sistema** (Windows, Samsung, iOS, Mac)	é a API `navigator.share`: a UI é do **SO**, não do app. Disponibilidade varia por navegador/SO
Barra do navegador na cor do app — só em alguns	`theme_color` do manifest: **cada navegador decide** (Firefox/Brave no desktop não pintam)
Android/Chrome: só **"Criar atalho"**, sem **"Instalar"**	critérios de instalabilidade: manifest **sem ícones 192/512**. Firefox e Safari abriram em tela cheia mesmo assim
Outro site, sem SW, **falha offline**	ter botão "instalar" ≠ ser PWA — e ser PWA ≠ ter botão
Quase ninguém distinguiu PWA instalado de **app nativo**	`display: standalone` esconde a barra: parece app

> "Instalar" é decisão do **navegador + sistema**, e quando falha **não há log**. Em produção isso vira "no meu aparelho não funciona": é a complexidade que um app de loja não tem.

Site, PWA ou app nativo com PWA dentro? — o que cada um dá

	Site no navegador	PWA (instalada)	PWA dentro de app nativo (WebView)
Distribuição	URL	URL + ícone na tela	**App Store / Google Play**
Instalação	—	manifest + critérios do navegador	pela loja (manifest e `beforeinstallprompt` **não se aplicam**)
Offline (Service Worker)	só se tiver SW	sim	Android: em geral sim · **iOS (WKWebView): só com *App-Bound Domains***
APIs	só Web APIs	só Web APIs	Web APIs **+ ponte nativa** (biometria, push nativo, Bluetooth…)
Atualizar	deploy	deploy	conteúdo web: deploy · casca nativa: loja
Risco	—	descoberta por URL/busca	Apple **4.2**: app que é "só um site" pode ser rejeitado

> **Vantagem do PWA na WebView sobre o site comum:** presença na loja + acesso nativo, **sem reescrever** a interface. **Custo:** uma casca para manter; regras da Apple; SW no iOS exige configuração. Se você só precisa de ícone e offline, o PWA instalado já basta.

PWA dentro do nativo — como é no código

```dart
// Flutter: a WebView abre o PWA
import 'package:webview_flutter/webview_flutter.dart';

WebViewController()
  ..setJavaScriptMode(JavaScriptMode.unrestricted)
  ..loadRequest(Uri.parse('https://pwa-tmdb-aula4.vercel.app'));
```

```kotlin
// Android (Kotlin): mesma ideia
val webView = WebView(context)
webView.settings.javaScriptEnabled = true
webView.loadUrl("https://pwa-tmdb-aula4.vercel.app")
```

> **Hybrid shell:** app nativo na loja + PWA dentro da WebView. O **Ionic Capacitor** (capacitorjs.com) formalizou o padrão para devs web; `webview_flutter` é o equivalente Flutter. "Adoção ampla" é afirmação do próprio fabricante — leia como dado de fornecedor.

TWA — a alternativa Android à WebView

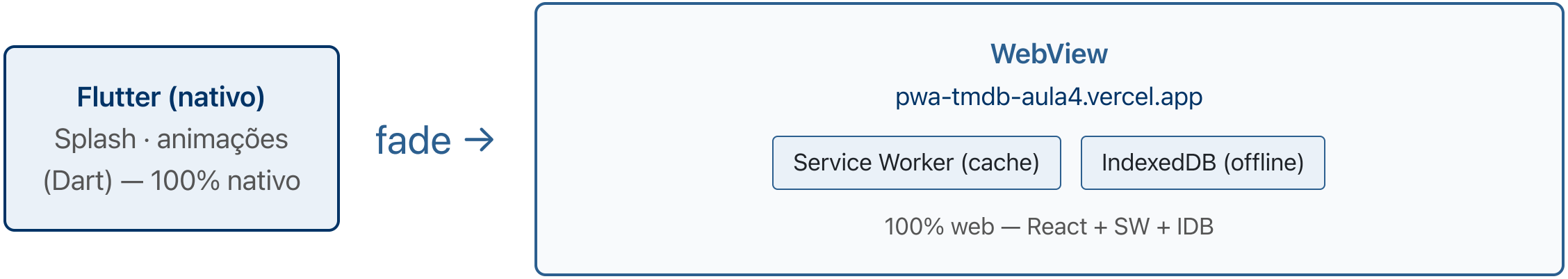
	WebView	TWA (Trusted Web Activity)
Quem renderiza	componente embutido no app	o **Chrome** (sem barra de endereço)
Play Store	sim	**sim**
iOS	sim (WKWebView)	**não** — a Apple não suporta
Login com Google	bloqueado em WebView embutida	funciona (roda no navegador de verdade)
Requisito	código nativo da casca	PWA válida + vínculo do domínio ao app (*Digital Asset Links*)

> **Trade-off:** TWA melhora a experiência no Android, mas você passa a manter **uma casca só de Android**. Se não precisa disso, o PWA instalado já resolve. *(Tópico trazido por um aluno em aula.)*

E se quiser os dois? Flutter + PWA

A escolha **não é binária**. Padrão real usado em produção:

Flutter (ou RN) como shell → WebView carregando o PWA



Usuário não sabe que saiu do Flutter — UA: `CineHub/1.0 FlutterShell`

> O User-Agent `CineHub/1.0 FlutterShell` permite ao PWA detectar o contexto e adaptar comportamento.

> **Ionic Capacitor** (capacitorjs.com) popularizou este padrão para devs web. O demo usa a mesma arquitetura via `webview_flutter`.

Demo (professor) — Flutter Shell abrindo o PWA

```
cd exercicios/04-pwa-tmdb/demo/flutter-pwa-bridge
flutter pub get
flutter run        # celular ou emulador — a WebView não roda em -d chrome
```

O que observar:

1. Splash Flutter (nativo) → fade para a WebView
2. O PWA carrega dentro do app, **sem barra de endereço**
3. ✈️ Modo avião → o app ainda abre? (depende do Service Worker: Android em geral sim; iOS precisa de *App-Bound Domains*)
4. Debug: Android → `chrome://inspect` · iOS → Safari → Develop

> **Pergunta para a turma:** o que esse app ganhou em relação a abrir a mesma URL no Chrome? E o que **não** ganhou?

Anatomia de uma PWA

HTTPS

Origem segura obrigatória — SW só registra em HTTPS (ou localhost)

Service Worker

Proxy de rede — intercepta fetch, gerencia cache, habilita offline

Web App Manifest

Metadados: nome, ícones, cores, `display: standalone`

Precisa dos 3 para ser instalável. HTTPS + SW = offline. Manifest = "cara de app nativo".

De graça: o mesmo manifest instala em mobile *e* desktop (Chrome/Edge) — `beforeinstallprompt` dispara nos dois, sem código a mais.

Por que o PWA funciona offline?

> "Para exibir qualquer coisa, o navegador precisa de HTML, CSS e JS. A mágica do PWA não é fazer esses arquivos aparecerem do nada — é **como ele os armazena no seu dispositivo**."

O segredo tem nome: Service Worker



Decide em runtime: *"pego da internet ou sirvo do cache?"*

Como o PWA funciona offline — passo a passo

1

1º acesso (com internet)
Browser baixa HTML, CSS, JS, imagens + instala o Service Worker

→

2

Cache Storage
SW copia assets críticos pra uma "despensa" local no navegador

→

3

Offline ✈️
SW intercepta requisições → serve arquivos da "despensa" — sem rede

> App **abre instantaneamente** — navegador lê HTML e JS do cache local.

E os dados dinâmicos? — IndexedDB

Cache Storage (Service Worker)

HTML · CSS · JS · imagens — o *esqueleto* do app

← instalação / 1ª visita

+

IndexedDB (idb)

Filmes · favoritos · posts — o *conteúdo* do usuário

← ao usar o app online

No A4: SW serve o app → React lê filmes do IndexedDB → funciona offline sem precisar de rede. ✓

Web App Manifest

```
{
  "name": "Minha PWA",
  "short_name": "MinhaApp",
  "description": "Descrição curta da app",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#003366",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

Instalável de verdade: critérios + `beforeinstallprompt`

Para o Chrome/Edge oferecer "Instalar": HTTPS · manifest válido (`name` , `icons` 192 e 512, `start_url` , `display: standalone`). Instalar pelo **menu** não exige mais SW (Chrome 108 mobile / 112 desktop); o **prompt** (`beforeinstallprompt`) ainda exige um SW com handler de `fetch` .

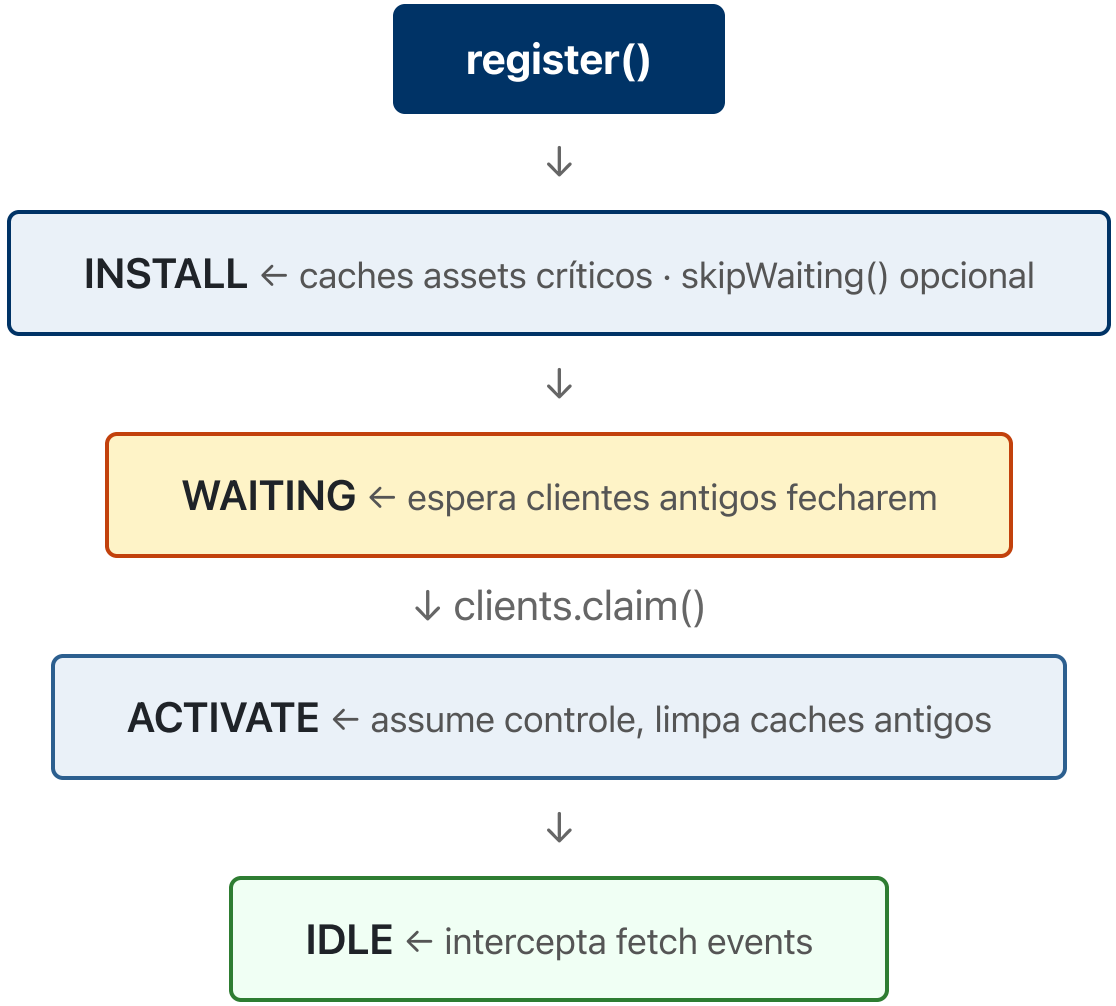
```
let deferredPrompt: any = null;

window.addEventListener('beforeinstallprompt', (e) => {
  e.preventDefault();          // segura o banner automático
  deferredPrompt = e;          // guarda pra disparar no SEU botão
  setCanInstall(true);
});

async function install() {
  deferredPrompt.prompt();
  const { outcome } = await deferredPrompt.userChoice;  // 'accepted' | 'dismissed'
  deferredPrompt = null;
}
window.addEventListener('appinstalled', () => setCanInstall(false));
```

> É o que o `InstallButton` da A4 já faz — funciona em **desktop e mobile**. **iOS/Safari não dispara `beforeinstallprompt`** : o usuário usa *Compartilhar → Adicionar à Tela de Início*.

Service Worker — lifecycle



Service Worker — install + activate

```
// sw.js
const CACHE_NAME = 'app-v3';
const PRECACHE_URLS = ['/', '/index.html', '/styles.css', '/app.js'];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => cache.addAll(PRECACHE_URLS))
  );
  self.skipWaiting(); // assume controle imediato (cuidado!)
});

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((names) =>
      Promise.all(
        names.filter((n) => n !== CACHE_NAME).map((n) => caches.delete(n))
      )
    )
  );
  self.clients.claim();
});
```

Atenção: *ativado* não é *no controle*

```
// 1ª visita: o SW instala e ativa, mas a página que carregou
// ANTES dele NÃO é controlada por ele.
navigator.serviceWorker.controller   // → null na 1ª visita!

self.addEventListener('activate', (e) => e.waitUntil(self.clients.claim()));
// clients.claim() → assume as abas já abertas sem precisar recarregar
```

Situação	O que acontece
1ª visita, sem `clients.claim()`	offline **só funciona depois de recarregar**
Nova versão do SW	fica em **waiting** até as abas antigas fecharem
`skipWaiting()`	pula a espera — mas pode misturar JS novo com cache velho

> Bug clássico de "funciona no meu PC": você testa na 2ª visita; o usuário perde o offline na 1ª.

"Publiquei a v2 e não vejo a mudança" — o SW *waiting*

```
1. Você abre o site        → o SW ANTIGO responde (HTML/JS vêm do precache)
2. Navegador baixa sw.js   → mudou? instala o SW NOVO ao lado do antigo
3. SW novo fica "waiting"  → espera as abas antigas fecharem
4. Só então ele assume     → a tela nova aparece no PRÓXIMO carregamento
```

`registerType`	Quem tira o SW novo do *waiting*	O usuário percebe?
`'autoUpdate'`	o próprio SW (`skipWaiting`)	não — muda no reload seguinte, sem aviso
`'prompt'`	**o usuário**, clicando em *Atualizar*	sim — toast "Nova versão disponível"

- **Aba anônima funciona** porque não tem SW antigo: busca tudo na rede.
- Preso numa versão velha? DevTools → Application → **Service Workers** → *Unregister* + **Clear site data** → ⇧⌘R. (Em dev: marque *Update on reload*.)

> O toast só aparece pra quem **já tinha um SW com o `UpdatePrompt`**. Quem abriu uma versão sem ele pode ficar preso sem saber — é por isso que o modo `prompt` existe.

Service Worker — anatomia do `fetch` event

```
self.addEventListener('fetch', (event) => {     // ①
  if (event.request.url.includes('/static/')) { // ②
    event.respondWith(                          // ③
      caches.match(event.request).then((cached) => { // ④
        return cached || fetch(event.request).then((res) => { // ⑤
          caches.open(CACHE).then(c => c.put(event.request, res.clone())); // ⑥
          return res;
        });
      })
    );
  }
});
```

① self.addEventListener('fetch')

Dispara em *toda* req do app — HTML, CSS, JS, API. SW age como proxy.

② if ('/static/')

Roteador: estratégia diferente por tipo de URL (Cache-first p/ assets, Network-first p/ API).

③ event.respondWith()

"Sequestra" o fluxo. Navegador espera *você* entregar a resposta — cache ou rede.

④ caches.match()

Vasculha Cache Storage. Retorna arquivo salvo ou `undefined` se não cacheado ainda.

⑤ cached || fetch()

Cache hit → serve imediato (offline ✓). Cache miss → vai à rede e cacheia pra próxima.

⑥ res.clone()

Body é stream único — lê só uma vez. Clone vai pro cache; original vai pro navegador.

Cache strategies

Cache-first (estáticos)

Tenta cache; se falhar, vai pra rede; cacheia o resultado.

Network-first (dados dinâmicos)

Tenta rede com timeout; se falhar, vai pro cache.

Stale-While-Revalidate

Serve cache imediatamente (rápido) + busca rede em paralelo (atualiza próxima request).

Cache-only / Network-only

Casos específicos (assets imutáveis vs telemetria).

Stale-While-Revalidate — implementação

```
self.addEventListener('fetch', (event) => {
  if (event.request.url.includes('/api/feed')) {
    event.respondWith(
      caches.open(CACHE_NAME).then(async (cache) => {
        const cached = await cache.match(event.request);
        const fetchPromise = fetch(event.request).then((response) => {
          cache.put(event.request, response.clone());
          return response;
        });
        return cached || fetchPromise;
      })
    );
  }
});
```

> Usuário vê algo imediatamente, próxima carga já está atualizada.

A4 — Começando (clone novo, sem conflito)

```
# 1. Clone o repo do professor numa pasta nova
git clone https://github.com/jacksonsmith/puc-iec-mobile-multiplataforma.git a4-pwa
cd a4-pwa

# 2. Crie a branch da atividade
git switch -c feat/a4-pwa

# 3. Aponte o remote "meu" para o SEU fork e envie a branch
git remote add meu https://github.com/<seu-login>/puc-iec-mobile-multiplataforma.git
git push -u meu feat/a4-pwa
```

> **Não precisa sincronizar fork nem resolver conflito.** Clone novo = tudo atualizado. Depois: PR da branch `feat/a4-pwa` do seu fork → `main` do professor. Windows: Git Bash ou WSL.

A4 — Setup (3 passos)

```
# Passo 0 — confirmar que está na pasta certa
cd exercicios/04-pwa-tmdb/pratica
ls src/services          # deve listar: tmdb.ts  db.ts
```

> **Windows:** `cd exercicios\04-pwa-tmdb\pratica` e `dir src\services`

```
# Passo 1 — instalar dependências
npm install

# Passo 2 — configurar token TMDB
cp .env.example .env.local        # Windows cmd: copy .env.example .env.local
# edite .env.local → VITE_TMDB_TOKEN=<seu-token-aqui>
# Gere em: themoviedb.org/settings/api → API Read Access Token

# Passo 3 — rodar
npm run dev           # dev server: http://localhost:5173
```

A4 — O que implementar (TODO 1)

Arquivo: `src/services/tmdb.ts` — hoje a função só lança erro:

```
export async function fetchPopularMovies(_page = 1): Promise<MoviesResponse> {
  throw new Error('TODO 1: fetchPopularMovies não implementada');
}
```

Spec: `GET /movie/popular` com `params: { language: 'pt-BR', page }` e retornar o `data` da resposta.

> `tmdbClient` é um axios já configurado (`baseURL` + `Authorization: Bearer`). **Em aula você faz este TODO sozinho** — a verificação é `npm test` ficar verde.

TODO 1 — resultado esperado

Antes: a tela mostra o erro `TODO 1: fetchPopularMovies não implementada` e `npm test` tem 2 testes vermelhos.

Depois: a lista de **filmes populares** aparece (pôster, título, nota) e:

```
npm test   →   ✓ 1. retorna lista de filmes via tmdbClient
               ✓ 2. propaga erro quando tmdbClient lança
```

Sinal de pronto: lista na tela **e** `02-tmdb-service.test.ts` verde.

> Sem token? A tela mostra "token ausente" — volte ao passo 2 do setup (`.env.local`).



TODO 1 — 3 erros comuns (vistos em aula)

```
// ❌ query params soltos: o axios IGNORA — o 2º argumento é a CONFIG
tmdbClient.get('/movie/popular', { language: 'pt-BR', page })

// ❌ sem o tipo: `data` vira o genérico T → "Type 'T' is not assignable to 'MoviesResponse'"
const { data } = await tmdbClient.get('/movie/popular', { params: { language: 'pt-BR', page } })

// ✅
const { data } = await tmdbClient.get<MoviesResponse>('/movie/popular', {
  params: { language: 'pt-BR', page },
});
return data;                          // ← o `data`, não a resposta inteira do axios
```

Também derruba o teste 1: esquecer `language: 'pt-BR'` nos `params`.

⚠️ **O app pode funcionar mesmo com o TODO 1 errado ou vazio.** O `networkFirst` captura o erro e lê o **IndexedDB** de uma execução anterior. Antes de conferir: DevTools → Application → **Clear site data** → recarregar. Sem cache e sem o TODO, a tela fica vazia ou com erro — é o teste de verdade.

☕ Intervalo — 30min

Volta às :___

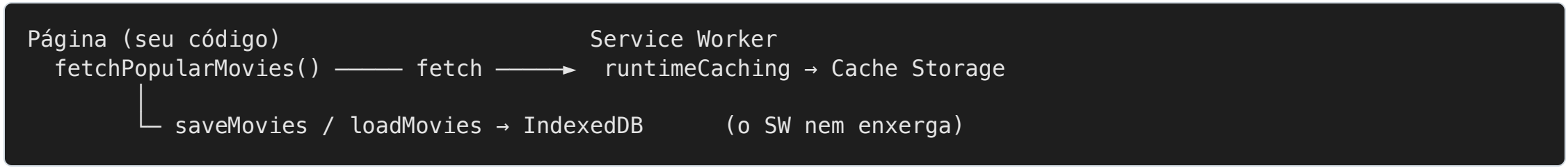
Aproveita pra:

- Esticar perna
- Pegar token TMDB se não tiver: developer.themoviedb.org → API → Bearer
- Abrir Chrome DevTools → aba **Application**

> Próximo bloco: PWA com Service Workers. Chrome DevTools aberto na aba **Application**.

Duas camadas de cache — quem guarda o quê

	Service Worker (`runtimeCaching`)	IndexedDB (`idb`)
Roda em	o **SW** (fora da página)	o **seu código** (na página)
Guarda	resposta **HTTP bruta** (headers + body)	**objetos JS** já tratados (`results[]`)
Onde aparece no DevTools	Application → **Cache Storage** (`tmdb-api` , `tmdb-images`)	Application → **IndexedDB** → `tmdb-cache`
Quem decide	regra por **URL**, sem código por tela	você chama `saveMovies` / `loadMovies`
Imagens	✅ guarda os pôsteres	❌ só a **URL** do pôster



> Os dois **não se conhecem**: são armazenamentos separados, com ciclos de vida separados.

Se já tenho IndexedDB, por que preciso do `runtimeCaching`?

Para a API de filmes, as duas se sobrepõem — a mesma lista fica guardada em dois lugares. O que o IndexedDB **não** resolve sozinho:

1. **Os pôsteres** — o IDB guarda só a URL; offline, a lista aparece com imagem quebrada. A regra `CacheFirst` de `image.tmdb.org` é a que mais faz diferença.
2. **O próprio app** — pra ler o IDB, o JS precisa carregar. Quem entrega HTML/JS offline é o **precache** do SW.
3. **Instalar** — PWA instalável exige SW + manifest.

⚠️ **Pegadinha:** offline de verdade, o `fetch` da API passa pelo SW, que responde do cache (na hora se a rede falha; após 3 s se ela pendura). A chamada **não falha** → o `loadMovies` do `networkFirst` nunca roda. O IDB só aparece quando a request **lança erro** (ex.: o simulador de offline do app).

> Pergunta pra pensar: *se eu apagar o Cache Storage mas manter o IndexedDB, o que acontece offline?*

Quando usar cada um

	`runtimeCaching` (Cache Storage)	IndexedDB
Escopo	**toda** a API, por padrão de URL — "rede de proteção"	só o que **seu código** salva
Rota nova, sem código nenhum	já fica offline sozinha ✅	❌ ninguém chamou `saveX`
Dado transformado / do usuário (comentário, "já assisti")	❌ é a resposta bruta	✅ independe da API
Vida útil	**expira** (`maxAgeSeconds` , `maxEntries`)	o tempo que você quiser
Mudou o formato dos dados	—	✅ `version` + migração de schema

> Cache Storage = *arquivos e respostas HTTP brutas* (como um **Redis offline** de respostas). IndexedDB = *banco* local, com chave/valor e índices. **Com rede, o `NetworkFirst` sempre devolve a resposta nova**; o cache só entra quando a rede falha ou passa do timeout.

O que é o Workbox?

Workbox é uma biblioteca do Google que abstrai a API de Service Worker.

Sem Workbox, você escreve cache manual para cada rota:

```
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached => cached || fetch(event.request))
  );
});
```

Com Workbox, você declara a estratégia:

```
registerRoute(/\.png$/, new CacheFirst());
```

Sem Workbox	Com Workbox
~150 linhas de boilerplate	~10 linhas declarativas

> Workbox é para Service Workers o que o Axios é para `fetch` — você podia fazer sem, mas não vale a pena.

Workbox — abstração madura

```
import { precacheAndRoute } from 'workbox-precaching';
import { registerRoute } from 'workbox-routing';
import {
  CacheFirst,
  NetworkFirst,
  StaleWhileRevalidate,
} from 'workbox-strategies';

precacheAndRoute(self.__WB_MANIFEST);

registerRoute(/\.(?:png|jpg|svg)$/, new CacheFirst({ cacheName: 'images' }));
registerRoute(
  /\/api\/static\//,
  new StaleWhileRevalidate({ cacheName: 'api-static' })
);
registerRoute(/\/api\/dynamic\//, new NetworkFirst({ cacheName: 'api-dynamic' }));
```

> Workbox elimina boilerplate. Use em produção.

IndexedDB — persistência local estruturada

`localStorage` é síncrono e limitado (~5MB). **IndexedDB** é a opção real.

```
import Dexie from 'dexie';

const db = new Dexie('MyAppDB');
db.version(1).stores({
  drafts: '++id, content, createdAt',
  user: '&id, name, email',
});

// Add
await db.drafts.add({ content: 'Hello', createdAt: Date.now() });

// Query
const recent = await db.drafts
  .orderBy('createdAt')
  .reverse()
  .limit(10)
  .toArray();
```

> Dexie torna IndexedDB usável.

Background Sync API

Usuário fecha app sem conexão. Operação fica pendente. Quando conectividade volta, **Service Worker dispara `sync` event mesmo com app fechado**.

```
// Page side
async function submitOfflineSafe(message) {
  await db.outbox.add({ message, attempts: 0 });

  if ('serviceWorker' in navigator && 'SyncManager' in window) {
    const registration = await navigator.serviceWorker.ready;
    await registration.sync.register('flush-outbox');
  }
}

// sw.js
self.addEventListener('sync', (event) => {
  if (event.tag === 'flush-outbox') {
    event.waitUntil(flushOutbox());
  }
});
```

Web Worker vs Service Worker — quando usar cada um?

Web Worker — "preciso processar dados sem travar a UI"

```
// Parsing de CSV gigante sem congelar a página
const worker = new Worker('parser.js');
worker.postMessage(csvData);          // envia pra thread separada
worker.onmessage = e => render(e.data); // recebe resultado
```

🎯 Casos: filtrar 50k registros, aplicar filtro em imagem, rodar modelo ML

Service Worker — "preciso interceptar rede e controlar cache"

```
// Responde a fetch do app inteiro, mesmo sem internet
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request) || fetch(event.request)
  );
});
```

🎯 Casos: carregar app offline, cachear TMDB, push notification, sync em background

> Web Worker = **thread paralela** pra sua página.
> Service Worker = **proxy de rede** que persiste entre sessões.

`vite-plugin-pwa` — como o Workbox é configurado

O plugin gera o Service Worker a partir do `vite.config.ts`. Você **não escreve `sw.js` à mão**: descreve as regras.

```
// vite.config.ts — exemplo com OUTRA API (na A4 você escreve as regras do TMDB)
VitePWA({
  workbox: {
    runtimeCaching: [
      {
        urlPattern: /^https:\/\/api\.exemplo\.com\//,
        handler: 'NetworkFirst',            // dados: rede primeiro, cache de reserva
        options: {
          cacheName: 'api-exemplo',
          networkTimeoutSeconds: 3,         // rede lenta? desiste e usa o cache
          expiration: { maxEntries: 50, maxAgeSeconds: 60 * 60 * 24 },
        },
      },
    ],
  },
})
```

> **Precisa de framework?** Não — HTML/CSS/JS puros viram PWA. Na prática ninguém configura à mão: **Vite** → `vite-plugin-pwa` · **Angular** → `ng add @angular/pwa` · **Next** → libs da comunidade (ex.: Serwist).

> **Imagem cross-origin** = resposta *opaca* (status 0) → `cacheableResponse: { statuses: [0, 200] }`. **SW não roda no `dev`**: use `build` + `preview`.

Nova versão: como o usuário recebe? (decisão de arquitetura)

`registerType`	Comportamento	Quando usar
`'autoUpdate'` (estado inicial da A4)	baixa e **aplica sozinho**; recarrega na próxima navegação	conteúdo, apps sem estado sensível
`'prompt'`	avisa e **o usuário decide** quando atualizar	formulários longos, checkout, editor

```
import { useRegisterSW } from 'virtual:pwa-register/react';

const { needRefresh: [needRefresh], updateServiceWorker } = useRegisterSW();

{needRefresh && (
  <button onClick={() => updateServiceWorker(true)}>
    Nova versão disponível — atualizar
  </button>
)}
```

> Atualizar no meio de um formulário perde dados. Em app com estado local, prefira `prompt`.

Caso relatado em aula: hotfix publicado, cliente continuava na versão velha → o time pedia para **limpar o cache**... e acabou **removendo o PWA**. O deploy entrega o arquivo novo; **quem aplica é o SW novo, quando assume**. Atualização é decisão de arquitetura, não detalhe.

PWA — o que funciona (casos reais)

> Todos os números abaixo são dados de engenharia publicados pelas próprias empresas — leia com olhos críticos: são cases de sucesso, não estudos controlados.

Pinterest *(web.dev case study, 2017)*
→ 60% mais engajamento · 44% mais receita de anúncios · 40% menos tempo de interação ↓

AliExpress *(web.dev case study, 2016)*
→ 104% mais conversão em novos usuários · 74% mais tempo de sessão no iOS

Twitter Lite *(Eng Blog, 2017 — arquivado)*
→ 65% mais páginas/sessão · 75% mais tweets enviados · bounce rate –20%

> Padrão: são casos de *mobile web* melhorado (não nativo substituído). **Atenção à data:** os cases clássicos são de 2016–2017 e vêm dos próprios times. O que mudou de lá pra cá é a **plataforma** (slide a seguir): push no iOS, web app por padrão no iOS 26, critérios de instalação do Chrome.

PWA — o que mudou na plataforma (2023–2025)

iOS/Safari — o gap diminuiu, mas existe:

- **iOS 16.4 (2023):** Web Push chega — **só** para PWA instalada na Tela de Início, e com gesto do usuário para pedir permissão
- **iOS 26 (2025):** todo site adicionado à Tela de Início **abre como web app por padrão** (o usuário pode desligar) — o manifest continua valendo (WebKit, jun/2025)
- iOS **não dispara** `beforeinstallprompt`: instalar é *Compartilhar → Adicionar à Tela de Início*

Chrome: instalar pelo menu não exige mais SW (108/112); o prompt ainda exige handler de `fetch`.

Distribuição e capacidades:

- Sem App Store: descoberta depende de URL/busca
- Bluetooth, NFC, contatos, câmera avançada → ainda limitados/variam por navegador

> **Fontes:** WebKit blog (WWDC25) · Chrome for Developers — *Revisiting Chrome's installability criteria* · MDN.
> Confira a tabela de compatibilidade do MDN antes de decidir — suporte muda a cada versão.

PWA vs Nativo — quando escolher

✅ PWA faz sentido

- Usuário base mobile-web
- Conteúdo editorial / e-commerce leve
- Deploy instantâneo sem store review
- Cross-platform com um codebase
- App secundário / fluxo isolado

❌ Nativo faz mais sentido

- Bluetooth, NFC, ARKit/ARCore
- Push/background no iOS onde a UX precisa ser garantida (hoje só PWA instalada)
- App Store discovery é parte do produto
- Gaming ou AR/VR intensivo
- Integração deep com SO (widgets, live activities)

React Native for Web — o que é?

Criado por: Nicolas Gallagher no Twitter/Meta · **Lançamento:** 2018 · necolas.github.io/react-native-web

Proposta: rodar componentes React Native no browser sem reescrever — `View`, `Text`, `TouchableOpacity` viram `div`, `span`, `button` via polyfills DOM.

```tsx
// O mesmo componente funciona em RN (iOS/Android) e na web
import { View, Text } from 'react-native';

export function Card({ title }: { title: string }) {
  return <View><Text>{title}</Text></View>;  // web → <div><span>
}
```

Origem e uso: nasceu no Twitter (Twitter Lite, 2017, foi construído com ele) · hoje é a base do alvo web do Expo.

> **Problema real:** componentes pesos pesados do RN (Animated, FlatList) têm overhead no browser. Bundle maior que uma SPA pura.

React Native for Web — quando faz sentido?

Faz sentido quando:

- Já tem app RN e quer versão web **sem segunda codebase**
- Time conhece RN mas não React puro
- Componentes de negócio são simples (formulários, listas, texto)

Não faz sentido quando:

- Experiência web precisa de SEO robusto (SSR difícil)
- App tem UI muito nativa (gestos complexos, animações Reanimated pesadas)
- Projeto novo — prefira React + PWA direto

> **Posição no mercado (2026):** nicho. Poucos cases públicos nomeados em produção; adoção pequena vs React puro. Nós o apresentamos para você **reconhecer** a opção, não para recomendá-la como padrão.

Ionic Capacitor — o que é?

Criado por: time da Ionic Framework (Max Lynch, Ben Sperry) · **Lançamento:** 2019 · capacitorjs.com

Proposta: empacotar qualquer web app (PWA, React, Vue, Angular) como app nativo na App Store e Google Play, com acesso a APIs nativas via plugins JS.



Evolução do PhoneGap/Cordova — mesma ideia, mas com TypeScript, suporte a PWA, e integração moderna com Vite/React.

Ionic Capacitor — quando faz sentido?

Faz sentido quando:

- Já tem PWA ou web app e quer distribuir na store
- Precisa de câmera, push nativo, biometria, Bluetooth — sem abrir mão da web
- Time é 100% web, sem nenhum Swift/Kotlin

Não faz sentido quando:

- App precisa de performance gráfica (jogos, animações 60fps pesadas)
- Integração nativa muito profunda (sensores proprietários, ARKit/ARCore full)

> **Adoção real (2026):** larga — milhares de apps na store usam Capacitor. Empresa de saúde, fintech, B2B SaaS são os nichos mais comuns.

Web a partir de stack multiplataforma — estado em 2026

Opção	Estado	Bom para	Cuidado
PWA (React/Vue + `vite-plugin-pwa`)	maduro, amplo	web novo, conteúdo público, SEO	limites no iOS
Capacitor (web na loja)	usado (dado do fabricante)	quem tem web e quer loja	Apple 4.2
Flutter Web	estável	apps **logados**, dashboards, versão web de app Flutter	canvas → **SEO fraco**; download maior (CanvasKit ≈ 1,5 MB)
React Native Web (Expo)	estável, nicho	reaproveitar código de app RN	poucos cases públicos; SSR/SEO evoluindo
Compose Multiplatform Web (KMP)	**Beta** (set/2025)	acompanhar; protótipo	poucos cases; Kotlin/JS estável, Wasm Beta

> **Regra prática:** projeto web **novo** com SEO → web + PWA. **Reaproveitar time e código** de um app Flutter/RN/KMP → a versão web serve para a parte **logada**, não para páginas públicas. A coluna *Estado* mistura fontes de fabricante e blogs 2025–2026 (JetBrains, Expo, guias de Flutter Web): confira a fonte antes de decidir.

Atividade 4 — PWA: filmes instalável + offline (15 pts)

Stack: React + Vite + `vite-plugin-pwa` (Workbox) + IndexedDB (`idb`) + TMDB REST

#	Onde	O quê	Pts
1	`services/tmdb.ts`	`fetchPopularMovies()` — **em aula, sozinho**	2
2	`vite.config.ts`	`runtimeCaching` : API (NetworkFirst) + pôsteres (CacheFirst)	4
3	`vite.config.ts` + `UpdatePrompt.tsx`	`registerType: 'prompt'` + aviso "Nova versão"	3
4	`HomeScreen.tsx`	aba ★ Favoritos	2
5	`useOfflineStatus.ts`	online/offline + banner	1
—	`ENTREGA.md`	evidências + reflexão (manual)	3

TODO 2 — dois conceitos novos: de qual **host** vem cada recurso (descubra no Network) · o que é uma resposta **opaca** (próximo slide).

Entrega: clone novo → branch `feat/a4-pwa` → remote `meu` → PR. O bot lê seus arquivos e dá a nota mínima.

TODO 2 — dois conceitos antes de escrever a regra

1. Cada regra casa por URL → de qual host vem cada recurso?
DevTools → **Network** → recarregue → filtre **Fetch/XHR** (API) e **Img** (pôsteres) → coluna **Domain**. Se o `urlPattern` não casa, a regra nunca dispara e o Cache Storage fica **vazio** — é assim que você diagnostica.

2. Resposta opaca (status 0)
`<img>` de outra origem, sem CORS: o navegador **exibe**, mas o código **não lê** o status. O Workbox só guarda `200` por padrão → você escolhe quais status aceitar (`cacheableResponse`). Respostas opacas também podem consumir mais cota de armazenamento.

Recurso	Estratégia	Por quê
API de filmes	**NetworkFirst** (3 s)	dado muda; prefira o fresco, caia no cache se a rede falhar
Pôsteres	**CacheFirst**	imagem quase não muda; economiza rede

> Fonte: doc do Workbox — *Caching strategies* e `cacheableResponse` (links na atividade).

A4 — resultado esperado por TODO (como saber que terminou)

TODO	O que muda no app	Onde ver	Sinal de pronto
2 `runtimeCaching`	2ª visita mais rápida e **offline funciona**	Application → Cache Storage	`tmdb-api` **e** `tmdb-images` com itens; recarregar em **Offline** mostra a lista e os pôsteres
3 `prompt` + `UpdatePrompt`	o app **avisa** que há versão nova (não troca sozinho)	build novo → Application → Service Workers → **Update**	toast "Nova versão disponível" → **Atualizar** recarrega com a versão nova
4 Favoritos	aba ★ lista só os favoritados	tab bar (celular)	favoritar 2 filmes → aba ★ mostra 2 · sem favoritos → mensagem de estado vazio
5 `useOfflineStatus`	faixa laranja quando cai a rede	aba ★ + Network → Offline	`📡 Offline — exibindo dados do cache` aparece e some ao voltar
ENTREGA.md	—	`pratica/ENTREGA.md`	prints + reflexão preenchidos

> O bot confere por **leitura dos arquivos** (nota mínima); "funcionar de verdade" é o que você vê nesta tabela.

TODO 2 e 3 — como fica

Primeira visita (build + preview), depois de recarregar 1×:

- Toast **"App pronto para uso offline"** → o SW instalou e guardou o shell
- Cache Storage com `tmdb-api` e `tmdb-images`
- Offline + recarregar → **a lista continua abrindo**

Depois de um novo build + Update:

- Toast **"Nova versão disponível"** com **Atualizar / Depois**
- Atualizar → recarrega com a versão nova

> Se o toast não aparece: está no `npm run dev`? recarregou 1× antes? o `registerType` é `'prompt'`?



TODO 4 e 5 — como fica

Esquerda (TODO 4): aba ★ **Favoritos** com os filmes que você favoritou (badge **2** na tab bar).

Direita (TODO 5): com a rede em **Offline** aparece a faixa `📡 Offline — exibindo dados do cache` e os favoritos continuam visíveis.

Estado vazio: sem favoritos → *"Nenhum favorito ainda — toque ★ em um filme."*

> Em tela larga a tab bar não aparece (é só mobile): use o DevTools em modo celular (⌘⇧M / Ctrl+Shift+M).



Vite — comandos que você vai usar

Comando	O que faz
`npm run dev`	Servidor de desenvolvimento (hot reload)
`npm run build`	Build de produção (gera `dist/`)
`npm run preview`	Serve o build de `dist/` em http://localhost:4173
`npm test`	Roda testes unitários (Vitest)
`npm run test:watch`	Vitest em modo watch (re-roda ao salvar)

> Para testar o Service Worker offline você **precisa** do `build + preview` — o SW não roda em `dev`.

A4 — Rodando os testes

```
# Na pasta pratica/ (confirme com: ls src/services)
npm test
```

Saída esperada no scaffold (TODO 1 não implementado):

```
FAIL  src/__tests__/unit/02-tmdb-service.test.ts
  ✕ 1. retorna lista de filmes via tmdbClient
  ✕ 2. propaga erro quando tmdbClient lança
```

Saída esperada depois do TODO 1:

```
PASS  src/__tests__/unit/02-tmdb-service.test.ts
  ✓ 1. retorna lista de filmes via tmdbClient
  ✓ 2. propaga erro quando tmdbClient lança
```

> Teste vermelho = implementar. Verde = entregar.

A4 — Testando offline (demo ao vivo)

> **Demo rodando:** https://pwa-tmdb-aula4.vercel.app — abra antes de implementar.

No SEU build (o Service Worker só existe no build):

1. `npm run build && npm run preview` → http://localhost:4173
2. Deixe os filmes carregarem · DevTools → **Application → Cache Storage**: `tmdb-api` e `tmdb-images` devem ter itens
3. **Network → Offline** → recarregue → o app continua abrindo
4. Aba ★ **Favoritos** mostra o banner `📡 Offline` (TODO 5)
5. Nova versão (TODO 3): **recarregue 1×**, mude um texto, rode `build` de novo e clique **Application → Service Workers → Update** → aparece o aviso; **Atualizar** recarrega com a versão nova

> **Dois "offline" diferentes:** o SW serve o *app shell* e as respostas cacheadas; o **IndexedDB** (`moviesRepository`) serve a *lista*. Se o Cache Storage estiver vazio, o TODO 2 está errado — o IndexedDB sozinho não prova que o SW funciona.

Pitfalls comuns

❌ **`skipWaiting()` sem cuidado** — clientes abertos podem ficar inconsistentes
❌ **Cache infinito** — esquecer de limpar caches antigos no `activate`
❌ **`localStorage` ao invés de IndexedDB** — limite de 5MB e síncrono
❌ **Service Worker em scope errado** — não intercepta o que deveria
❌ **Não testar em "modo airplane"** — só descobre bug em produção
❌ **Assumir suporte iOS igual ao Chrome** — verificar sempre em Safari/iOS
❌ **Testar PWA no `npm run dev`** — o SW só roda no build + preview
❌ **Host errado no `urlPattern`** — a regra nunca casa e nada é cacheado (confira no Network)
❌ **Testar só na 2ª visita** — 1ª visita não é controlada sem `clients.claim()`
❌ **`autoUpdate` em tela com formulário** — a atualização apaga o que o usuário digitou

Pitfalls da A4 — o que apareceu na aula

❌ **Conferir sem limpar o IndexedDB** — o app "funciona" por cache de uma rodada anterior
❌ `get(url, { language, page })` — params soltos são ignorados; use `{ params: {...} }`
❌ **Esquecer o `<MoviesResponse>` no `get`** — erro `Type 'T' is not assignable…`
❌ **Achar que IndexedDB e Cache Storage são a mesma coisa** — são separados (DevTools mostra em abas diferentes)
❌ **Testar a atualização numa aba que já tinha SW antigo** — o SW novo fica em *waiting*; teste também em aba anônima
❌ **`vite.config.ts` com sublinhado vermelho em `test`** — o `/// <reference types="vitest" />` precisa estar na **linha 1**; é só o editor, o `npm test` roda

Leituras obrigatórias (pré-Aula 5)

Públicas e gratuitas:

- **Archibald, J.** — *The Offline Cookbook* · web.dev/offline-cookbook · Google, 2014 (atualizado)
- **Workbox docs** — *Caching Strategies* · developer.chrome.com/docs/workbox/caching-strategies-overview
- **MDN** — *Service Worker API* · developer.mozilla.org/pt-BR/docs/Web/API/Service_Worker_API

Hybrid shell (padrão do demo):

- **Ionic** — *Capacitor: native runtime for web apps* · capacitorjs.com (mesmo padrão do demo; versão para devs web em vez de Flutter)
- **webview_flutter** — pub.dev/packages/webview_flutter (pacote usado no demo)

Leituras obrigatórias (pré-Aula 5) — continuação

Casos de uso (dados de indústria — leia criticamente):

- **Pinterest** PWA case study · web.archive.org — Google Showcase 2017
- **AliExpress** PWA case study · web.dev/aliexpress
- **Twitter Lite** Eng Blog 2017 · web.archive.org

Atualizadas (2023–2025) — recomendadas:

- **web.dev** — *Learn PWA* (curso gratuito, módulos *Update*, *Installation prompt*, *Offline data*) · web.dev/learn/pwa
- **Chrome for Developers** — *Revisiting Chrome's installability criteria* · developer.chrome.com/blog/update-install-criteria
- **WebKit blog** — *News from WWDC25: WebKit in Safari 26 beta* (web apps na Tela de Início) · webkit.org/blog/16993
- **vite-pwa** — docs do `vite-plugin-pwa` (registerType, `useRegisterSW`) · vite-pwa-org.netlify.app
- **JetBrains** — *Compose Multiplatform 1.9.0: Compose for Web goes Beta* · blog.jetbrains.com/kotlin/2025/09/compose-multiplatform-1-9-0-compose-for-web-beta
- **WebKit** — *App-Bound Domains* (Service Workers em WKWebView) · webkit.org/?p=10882
- **Expo** — *Expo Router* (web: static rendering / SSR) · docs.expo.dev/router

Próxima aula (26/10)

Aula 5 — KMP: Kotlin Multiplatform + BFF/GraphQL

Pré-requisito: Atividade 4 (PWA) entregue — ver Canvas. Android Studio (Kotlin Multiplatform plugin) instalado.

Prometido na Aula 3, entra na próxima:

- Segurança: **SSL pinning / man-in-the-middle** (KMP ou PWA)
- Experimento de **IA / ML na GPU (WebGPU)** com os filmes
- Projeto Final: **comecem a formar os grupos**

> KMP (Atividade 5) é feito **ao vivo na Aula 5** — não precisa configurar antes.

Obrigado!

📧 jackson.96@gmail.com
🔗 linkedin.com/in/3jacksonsmith
🐙 github.com/jacksonsmith

Próxima segunda, 26/10, 19h